Mini Project Report - Scientific Calculator with DevOps

By Chandrima Nandi (IMT2022062)

What is DevOps?

DevOps is a combination of **Development (Dev)** and **Operations (Ops)** that focuses on collaboration between software developers and IT operations teams. It aims to automate and integrate the processes of building, testing, and deploying software so that products can be delivered faster, more reliably, and with higher quality.

In a DevOps approach, tools like Jenkins, Docker, Ansible, and GitHub are used to automate the pipeline starting from code integration to deployment, enabling continuous integration (CI) and continuous delivery (CD).

Why DevOps?

Traditionally, development and operations teams worked separately where developers aim for rapid feature delivery, while operations focus on maintaining stability and up-time. This separation often caused delays, inconsistencies, and deployment issues.

DevOps solves these problems by:

- Bridging the gap between development and operations teams.
- Automating repetitive tasks such as building, testing, and deployment.
- Reducing time-to-market with faster and more reliable release cycles.
- Improving software quality through continuous integration and testing.
- Enhancing collaboration and accountability across the entire team.

Why DevOps for this project?

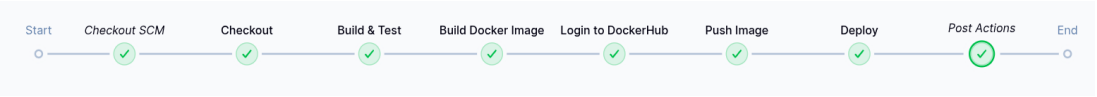
DevOps plays a crucial role in automating and managing the workflow for the Scientific Calculator CLI application. By integrating DevOps practices:

- The entire build, test, and deployment process of the Java-based calculator is automated using Jenkins pipelines.
- Docker ensures that the calculator runs in the same environment everywhere, eliminating the “works on my machine” issue.
- Ansible automates container deployment, ensuring smooth and consistent delivery of the application on any system.
- Continuous Integration (CI) allows the calculator’s functionality to be tested automatically after each commit, ensuring that every code update remains stable.

Tools Used

1. **GitHub (Source Control)** – Stores and manages code versions collaboratively; chosen for its simplicity and seamless Jenkins integration.
2. **Maven (Build and Test)** – Compiles and tests code efficiently; chosen for its strong dependency management and easy integration with Java projects.
3. **Jenkins (Continuous Integration)** – Automates the entire CI/CD pipeline; chosen for its flexibility and vast plugin ecosystem.
4. **Docker (Containerization)** – Packages the app and dependencies into a portable container; chosen for consistency across environments.
5. **Ansible (Deployment)** – Automates container deployment and configuration; chosen for its agentless, easy-to-manage infrastructure automation.

Jenkins Pipeline



1. Checkout SCM - Jenkins pulls the latest code from the GitHub repository to ensure the pipeline always runs on the most recent code.

```
stage('Checkout')
{
    steps
    {
```

```
    checkout scm
  }
}
```

2. Build & Test - (build) Maven compiles the source code into bytecode (`.class` files)

   (test) Maven runs unit tests (JUnit) to ensure your code works correctly.

```
stage('Build & Test')
{
   steps
   {
      sh 'mvn clean test'
   }
}
```

3. Build Docker Image - Docker reads the `Dockerfile` in the project folder and builds an image with the application and all dependencies.

```
stage('Build Docker Image')
{
   steps
   {
      sh 'docker build -t ${DOCKERHUB_REPO}:${IMAGE_TAG} .'
   }
}
```

4. Login to DockerHub - Jenkins securely logs into DockerHub using saved credentials.

```
stage('Login to DockerHub')
{
   steps
   {
      withCredentials([usernamePassword(credentialsId: 'dockerhub-cr
eds', usernameVariable: 'DOCKER_USER', passwordVariable: 'DOCKER_
PASS')])
      {
```

```
            sh 'echo $DOCKER_PASS | docker login -u $DOCKER_USER --p
assword-stdin'
        }
    }
}
```

5. Push Image - Pushes the built Docker image to the DockerHub repository.

```
stage('Push Image')
{
    steps
    {
        sh 'docker push ${DOCKERHUB_REPO}:${IMAGE_TAG}'
    }
}
```

6. Deploy - Jenkins runs the Ansible playbook. Ansible pulls the Docker image from DockerHub and starts a container on the target host (here, the local computer)

```
stage('Deploy')
{
    steps
    {
        sh 'ansible-playbook -i ansible/inventory.ini ansible/deploy.yml'
    }
}
```

7. Post Actions – Executes specific tasks after the pipeline stages finish. The success block sends a success email, the failure block sends a failure email, and the always block runs regardless of the outcome, here listing Docker images for verification.

```
post {
    success
    {
        mail to: 'chandrimanandi2004@gmail.com',
            subject: "Pipeline Success: ${currentBuild.fullDisplayName}",
```

```
            body: "Maven + Docker + Ansible pipeline succeeded!"
        }
        failure
        {
            mail to: 'chandrimanandi2004@gmail.com',
            subject: "Pipeline Failed: ${currentBuild.fullDisplayName}",
            body: "Pipeline failed! Check Jenkins logs."
        }
        always
        {
            sh 'docker images | head -n 20'
        }
    }
```

Github commits

The project is available at the link - https://github.com/ChandrimaNandi/Scientific_calculator

```
chandrima@chandrima-IdeaPad-5-Pro-14ITL6:~/Desktop/Academics/sem_7/SPE/MiniProj/calculator$ git log --
oneline
570d622 (HEAD -> main, origin/main) changed deploy.yml
4d462e2 changed the pomxml maven dependencies
bfad9cb added email id
90fa69f updated deploy.yml
025b030 updated README.md
7573992 changed the tag for docker image
8b0b71b Converted project to Java+Maven, updated Dockerfile, Jenkinsfile, Ansible
bc2a165 Rename jenkins file to Jenkinsfile for Jenkins pipeline
928172a Use image built locally
89d4829 Use local Docker image instead of pulling from Docker Hub
d4a9470 Initial commit: Add calculator project files
```

Working of Calculator (CLI)

Docker image was **pushed to Docker Hub using the Jenkins pipeline**.

TAG

9

Last pushed 6 minutes by chandrimanandi

```
docker pull chandrimanandi/calculator:
9
```

Digest	OS/ARCH	Last pull	Compressed size
c4c96fe7fa24	linux/amd64	less than 1 day	89.35 MB

```
chandrima@chandrima-IdeaPad-5-Pro-14ITL6:~$ sudo docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED        STATUS                          PORTS     NAMES
584d2f4b649f   chandrimanandi/calculator:9    "java -jar calculato…"   27 hours ago   Restarting (1) 54 seconds ago             calculator
f44eaaaa6e17   calculator:latest              "python ./calculator…"   2 days ago     Up 15 minutes                             scientific_calculator
```

`docker ps` shows the list of all running docker containers in the system

Steps to run the calculator locally:

1. **Pull the Docker image** from Docker Hub:

```
docker pull chandrimanandi/calculator:9
```

> Replace 9 with the tag/version of the image you pushed.

2. **Run the calculator container locally**:

```
docker run -it chandrimanandi/calculator:9
```

- The `it` flag makes the container **interactive**, allowing you to input values from the CLI.
- Once it runs, the CLI menu will appear for operations like:
  - Square root (√x)
  - Factorial (x!)
  - Natural logarithm (ln(x))
  - Power function (x^b)

```
==== Scientific Calculator ====
1. Square Root
2. Factorial
3. Natural Logarithm
4. Power
5. Exit
Enter choice: 1
Enter number: 100
√100.0 = 10.0

==== Scientific Calculator ====
1. Square Root
2. Factorial
3. Natural Logarithm
4. Power
5. Exit
Enter choice: 2
Enter integer: 12
12! = 479001600
```

```
==== Scientific Calculator ====
1. Square Root
2. Factorial
3. Natural Logarithm
4. Power
5. Exit
Enter choice: 3
Enter number: 2.75
ln(2.75) = 1.0116009116784799

==== Scientific Calculator ====
1. Square Root
2. Factorial
3. Natural Logarithm
4. Power
5. Exit
Enter choice: 4
Enter base: 0
Enter exponent: 2
0.0^2.0 = 0.0
```

Code

1. Ansible Playbook for Local Deployment

- Deploys the Calculator Docker container on the local machine using Ansible.
- Ensures Python Docker SDK (used by ansible) is installed to communicate with Docker.
- Pulls the Docker image from Docker Hub using the current build ID as the tag.
- Uses `java -jar` to execute the Java `.jar` file inside the container.

2. Calculator Java Code

- Implements a scientific calculator with 4 operations: square root, factorial, natural log, and power.
- Uses a menu-driven CLI for interactive input.
- Handles invalid input for negative numbers and zero (where appropriate).

3. JUnit Tests

- Tests all the calculator methods using JUnit 5.
- Ensures correctness of the calculations.

4. Dockerfile

- Stage 1: Uses Maven + Java 17 to build the `.jar` file from source code.
- Stage 2: Uses a smaller Java runtime image to run the final `.jar`, reducing image size and improving efficiency.
- Result: Produces a lightweight, self-contained Docker image of the calculator.

5. Jenkins Pipeline (Declarative)

- Automates the full DevOps pipeline:
  1. Checkout code from GitHub.
  2. Build and run tests with Maven.
  3. Build Docker image.
  4. Login and push Docker image to Docker Hub.
  5. Deploy the container locally using Ansible.
- Sends email notifications on success/failure.

6. Maven `pom.xml`

- Sets Java 17 for compilation.
- Includes JUnit 5 for testing.
- Uses Maven Shade plugin to create a fat `.jar` which is a single executable file bundling all dependencies, making the app portable and easy to run anywhere.